Chapter 1

Routines

1.1 RasterScan

The routine performs a two-dimensional raster scan by moving along a selected axis while acquiring a profile along the perpendicular axis at each scan position. For example, when the X axis is selected as the scan axis, the system moves along X according to the specified scan step and acquires a Z-profile at each X position. This approach is particularly useful for vertical coupling measurements, where the optical power distribution can be characterized as a function of the X and Z positions.

During the scan, the profiles acquired using the Suruga Seiki utilities can be saved for further analysis. The acquired data are then cleaned by removing invalid or irrelevant samples that could interfere with the generation of the heatmap. The processed profiles are interpolated onto a common grid and used to generate an interactive heatmap representing the measured power distribution.

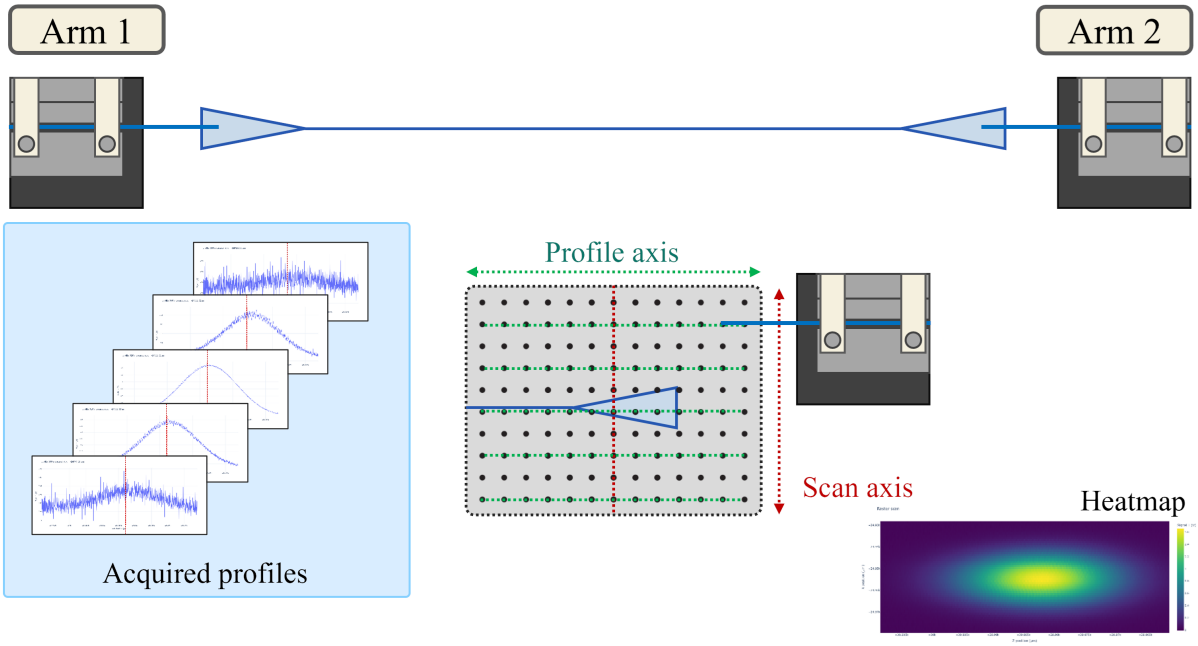

Figure 1.1: Two-dimensional raster scan for optical coupling localization. Profiles are acquired along the profile axis while the stage is incrementally moved along the perpendicular scan axis. The acquired profiles are combined to reconstruct a two-dimensional heatmap, allowing the position of maximum optical coupling to be identified.

Once the scan is complete, the routine identifies the global maximum power measured within the scanned area and, if enabled, automatically moves the arm to the corresponding X and Z position.

1.1.1 `clean_profile_data()`

Objective:

Clean and prepare the data acquired by the profile function for further processing.

This function receives the complete dictionary returned by `profile.get_profile_data()` and extracts only the variables required for the raster scan:

- `main_position`: the position of the axis being profiled.
- `signal1`: the measured optical signal.

The function then performs the following operations:

1. Converts the extracted data into NumPy arrays.
2. Removes invalid samples, including:
    - Non-finite position values.
    - Non-finite signal values.
    - Samples where the position is equal to zero.
3. Sorts and reorganizes the remaining samples according to their position.

The function **does not modify or save the original profile data**. Its purpose is to generate a cleaner and more manageable representation of the acquired data, which can then be used for further processing, particularly for generating the heatmap with `create_heatmap()`.

1.1.2 `save_profile_txt()`

Objective:

Preserve the complete profile data acquired during the measurement, together with the experimental parameters required to document and reproduce the measurement.

This function receives the complete dictionary returned by `profile.get_profile_data()`, including the following variables:

- `main_position`: position of the main axis during the profile.
- `sub1_position`: position of the first secondary axis.
- `sub2_position`: position of the second secondary axis.
- `signal1`: first measured signal.
- `signal2`: second measured signal.

In addition to the acquired data, the function constructs a header containing the experimental parameters and the stage positions at the time of acquisition:

- **Date:** Date and time of the measurement.
- **Center X:** X-axis center position.
- **Center Z:** Z-axis center position.
- **Arm:** Selected measurement arm or configuration.
- **Scan axis:** Axis along which the raster scan is performed.
- **Scan width:** Total width of the scan.
- **Scan step:** Distance between consecutive scan positions.
- **Profile axis:** Axis along which the profile is acquired.
- **Profile range:** Total range covered by the profile.
- **Profile speed:** Speed used during profile acquisition.

The function then stores the complete dataset and the associated metadata in a **tab-separated TXT file** using NumPy's `np.savetxt()` function.

Unlike `clean_profile_data()`, this function does **not** remove, filter, or reorganize the acquired samples. Its purpose is to preserve the original measurement data as completely as possible, while providing the experimental metadata necessary to identify and reproduce the measurement.

The resulting TXT file therefore contains both the **raw profile data** and the **measurement parameters** associated with its acquisition.

1.1.3 `create_heatmap()`

Objective:

Reconstruct a two-dimensional signal map from all the profiles acquired during the raster scan, generate an interactive heatmap, and determine the global maximum of `signal1` together with its corresponding X and Z coordinates.

The function receives two inputs:

- `scan_data`: a dictionary containing the processed profiles acquired during the raster scan. Each entry has the following structure:

```
[
    {
        "x": ...,
        "z_positions": ...,
        "signal": ...
    },
    ...
]
```

- `output_path`: the path where the generated heatmap will be saved.

Data Reconstruction

First, the function takes the Z positions from the first acquired profile and uses them as the common reference axis:

```
z_common = scan_data[0]["z_positions"]
```

Since the Z positions may not be identical for every profile, each profile is interpolated onto this common Z-axis. This ensures that all profiles have the same number of samples and can be represented consistently in a two-dimensional matrix.

For each profile, the function:

1. Retrieves the corresponding X position.
2. Retrieves the Z positions and measured signal.
3. Interpolates the signal onto `z_common`.
4. Adds the interpolated signal to the 2D signal matrix.

The resulting matrix represents the measured signal as a function of the X and Z coordinates and is used to generate the heatmap.

Heatmap Generation

The reconstructed 2D signal matrix is used to generate an **interactive Plotly heatmap**, where:

- The **X-axis** represents the raster scan position.
- The **Z-axis** represents the profile position.
- The **color scale** represents the measured `signal1`.

The resulting heatmap provides a two-dimensional visualization of the measured optical signal across the scanned area and is saved to the location specified by `output_path`.

Global Maximum Detection

In addition to generating the heatmap, the function determines the position of the maximum measured signal.

For each profile, the function first identifies its **local maximum**. The local maximum is then compared with the maximum found in the previous profiles. By continuously retaining the highest value, the function ultimately determines the **global maximum** across the entire raster scan.

The global maximum is stored together with its corresponding coordinates:

- **Maximum signal:** maximum value of `signal1`.
- **X coordinate:** X position of the profile where the maximum was found.
- **Z coordinate:** Z position corresponding to the maximum signal within that profile.

Therefore, `create_heatmap()` performs two main tasks: it **reconstructs and visualizes the complete raster scan as a two-dimensional signal map**, and it **identifies the position of the maximum output power** within the scanned region.

1.2 Automapper